

TDDMachine: generación automática del estado inicial de pruebas para Desarrollo Guiado por Pruebas mediante un modelo de lenguaje de gran escala

Gleiston Guerrero-Ulloa^{1,2} , Kerly M. Triana-Arrieta¹, Efraín Díaz-Macías¹ , Carlos Rodríguez-Domínguez^{2,3} , and Miguel J. Hornos² 

Abstract—The manual creation of test cases remains a laborious, repetitive and error-prone activity, and empirical evidence shows that developers test far less than they believe, treating testing as a low-value chore; yet Test-Driven Development (TDD), in which tests are written before production code, is associated with 40–90% lower defect density at the cost of only a modest increase in initial development effort. This paper presents TDDMachine, a Computer-Aided Software Engineering (CASE) tool that automates the generation of the initial test state (the *Red* stage of the TDD cycle) from structured specifications—functional requirements, use cases and user stories—using the Claude 3.5 Sonnet large language model. The requirements were derived from a systematic literature review of 108 primary studies; the technology stack (React 18, Django 4.2, PostgreSQL 15, Docker) was chosen through weighted multi-criteria evaluation; and generated `pytest` scripts are syntactically validated through Python Abstract Syntax Tree analysis. The tool was evaluated with 25 participants using the System Usability Scale (SUS) and the Technology Acceptance Model (TAM), and with 10 experts who assessed test quality through a structured rubric. TDDMachine produced valid scripts in 91.4% of attempts, reached a mean expert-rated quality of 4.08/5.0, obtained a SUS score of 74.46/100 (“Good”), and exceeded the technology-acceptance threshold in all four TAM constructs. The results provide empirical evidence that structured, LLM-assisted generation of the TDD initial state is both feasible and well accepted.

Index Terms—CASE tool; automated test generation; generative artificial intelligence; large language models; Test-Driven Development; `pytest`; software quality assurance; requirements engineering.

Resumen — La creación manual de casos de prueba sigue siendo una tarea laboriosa, repetitiva y propensa a errores, y la evidencia empírica muestra que los desarrolladores prueban mucho menos de lo que creen, percibiendo la escritura de pruebas como una carga de bajo valor; sin embargo, el Desarrollo Guiado por Pruebas (TDD), en el que las pruebas se escriben antes que el código de producción, se asocia con una reducción de la densidad de defectos del 40–90 % a cambio de un incremento moderado del esfuerzo inicial. Este artículo presenta TDDMachine, una herramienta CASE que automatiza la generación del estado inicial de las pruebas (la etapa *Red* del ciclo TDD) a partir de especificaciones estructuradas —requisitos funcionales, casos de uso e historias de usuario— mediante el modelo de lenguaje de gran escala Claude 3.5 Sonnet. Los requisitos se derivaron de una revisión

sistemática de 108 estudios primarios; la pila tecnológica (React 18, Django 4.2, PostgreSQL 15 y despliegue en contenedores Docker) se seleccionó mediante evaluación multicriterio ponderada; y los scripts `pytest` generados se validan sintácticamente mediante análisis del árbol sintáctico abstracto de Python. La herramienta se evaluó con 25 participantes mediante los instrumentos SUS y TAM, y con 10 expertos que valoraron la calidad de las pruebas mediante una rúbrica estructurada. TDDMachine produjo scripts válidos en el 91,4 % de los intentos, alcanzó una calidad media evaluada por expertos de 4,08/5,0, obtuvo un puntaje SUS de 74,46/100 (“Bueno”) y superó el umbral de aceptación tecnológica en los cuatro constructos TAM. Los resultados aportan evidencia empírica de que la generación estructurada del estado inicial TDD asistida por LLM es viable y bien aceptada.

Palabras Clave — herramienta CASE; generación automática de pruebas; inteligencia artificial generativa; modelos de lenguaje de gran escala; Desarrollo Guiado por Pruebas; `pytest`; aseguramiento de calidad; ingeniería de requisitos.

I. INTRODUCCIÓN

LAS pruebas de software constituyen una actividad transversal del ciclo de vida del desarrollo, responsable de verificar la funcionalidad, la estabilidad y la confiabilidad del código antes de su puesta en producción; sin embargo, su elaboración manual continúa siendo una tarea laboriosa, repetitiva y susceptible de errores, especialmente en entornos ágiles donde los ciclos de entrega son cortos y los requisitos cambian con frecuencia [1], [2].

Esta carga explica una tensión bien documentada en la práctica profesional: aunque las pruebas son reconocidas como esenciales, los desarrolladores tienden a percibir las como una actividad tediosa y de bajo valor inmediato, que compite con la escritura de funcionalidad. La evidencia empírica es contundente al respecto: el estudio de campo de Beller et al., basado en la instrumentación del entorno de desarrollo de miles de programadores, demostró que estos dedican a las pruebas mucho menos tiempo del que ellos mismos estiman, y que una fracción considerable de sesiones de desarrollo transcurre sin ejecutar una sola prueba [3]. En la misma línea, la encuesta de Straubinger y Fraser reporta que una proporción significativa de desarrolladores considera la escritura de pruebas como una labor poco atractiva y la posterga sistemáticamente frente a otras tareas [4]. En conjunto, estos trabajos sustentan la percepción, extendida en la industria, de que escribir pruebas es una inversión de tiempo cuyo retorno no siempre resulta evidente para quien las redacta.

La evidencia sobre resultados de proyecto, no obstante, apunta en sentido contrario cuando las pruebas se anticipan al código. El Desarrollo Guiado por Pruebas (*Test-Driven Development*, TDD) invierte el orden tradicional de codificación: primero se escribe una prueba que falla porque el código aún no existe (etapa *Red*), luego se implementa el código mínimo para que la prueba pase (etapa *Green*) y, por último, se refactoriza conservando las pruebas en verde (etapa *Refactor*) [5]. El estudio industrial de Nagappan et al. en

¹Facultad de Ciencias de la Computación, Universidad Técnica Estatal de Quevedo (UTEQ), Quevedo 120501, Los Ríos, Ecuador. ²Departamento de Ingeniería del Software, Universidad de Granada, 18071 Granada, España. ³Centro de Investigación de Tecnologías de la Información y Comunicación (CITIC), Universidad de Granada, 18071 Granada, España.

Autor de correspondencia: C. Rodríguez-Domínguez (correo: carlosrodriguez@ugr.es).

cuatro equipos de Microsoft e IBM documentó que la densidad de defectos previa a la liberación disminuyó entre un 40 % y un 90 % respecto de proyectos comparables que no aplicaron TDD, a cambio de un incremento del tiempo inicial de desarrollo de apenas 15 % a 35 % [6]. El metaanálisis de Rafique y Mišić confirma una mejora significativa de la calidad externa asociada al TDD, y el experimento controlado de Erdogmus et al. sobre el enfoque *test-first* evidenció ganancias de productividad atribuibles a la anticipación de las pruebas [7], [8]. La lectura conjunta de estos resultados es que un software construido a partir de sus pruebas tiene mayor probabilidad de cumplir las funcionalidades comprometidas dentro de los plazos previstos, y que la fase inicial de explotación —aquella en la que el equipo debe corregir defectos detectados tras la entrega— tiende a ser más reducida que en un software cuyas pruebas se escribieron después de construir el código.

El momento del ciclo de vida en que se escriben las pruebas es, por tanto, determinante. En el enfoque tradicional *test-last*, las pruebas se redactan una vez implementada la funcionalidad, con lo que actúan como verificación a posteriori y heredan las decisiones de diseño ya tomadas; en el enfoque *test-first* que promueve el TDD, en cambio, las pruebas se escriben antes del código de producción y operan como especificación ejecutable que guía el diseño [5], [8]. La ingeniería de requisitos ofrece un punto de partida natural para esta anticipación: cuando los requisitos funcionales, casos de uso e historias de usuario se documentan de forma estructurada conforme a estándares como ISO/IEC/IEEE 29148, sus criterios de aceptación son directamente transformables en condiciones de prueba verificables [9]. El reto práctico es que redactar esas pruebas iniciales desde requisitos, y hacerlo antes de escribir código, exige un esfuerzo cognitivo y una experiencia en *testing* de los que muchos equipos carecen, lo que limita la adopción del TDD pese a sus beneficios demostrados [10].

Ante retos de esta naturaleza, la ingeniería de software ha comenzado a incorporar inteligencia artificial generativa para automatizar tareas tradicionalmente manuales. Las revisiones sistemáticas más recientes documentan que los modelos de lenguaje de gran escala (*Large Language Models*, LLM) se aplican con métricas de éxito medibles a lo largo de numerosos procesos del ciclo de vida —generación y compleción de código, resumen y traducción de programas, reparación de defectos y elicitación de requisitos, entre otros— y no solo a la fase de pruebas [11], [12]. Esta tendencia se refleja también en la producción regional: en *Enfoque UTE* se han publicado revisiones y estudios que reportan la aplicación de técnicas de aprendizaje automático y profundo a problemas de ingeniería con indicadores cuantitativos de desempeño, desde la clasificación mediante redes neuronales artificiales [13] hasta el ajuste automático de ancho de banda en redes inalámbricas [14] o la integración de IoT e IA en la agricultura [15]. En el ámbito específico del *testing*, los LLM han mostrado capacidad para interpretar especificaciones en lenguaje natural y sintetizar casos de prueba estructurados y ejecutables, con tasas de éxito reportadas que varían según la calidad de la especificación de entrada [2], [16]–[20]. En la intersección específica entre el TDD y los LLM, la investigación reciente se ha concentrado sobre todo en el sentido inverso al que aborda este trabajo —generar el código de producción a partir de pruebas ya existentes, como en el enfoque *test-driven* para la generación de código de Mathews y Nagappan [21], o estudiar de forma exploratoria la conducción del ciclo TDD mediante asistentes conversacionales [22], [23]—, de modo que en esos trabajos las pruebas se presuponen dadas, mientras que TDDMachine se ocupa precisamente de generarlas en la etapa *Red* antes de que exista código de producción.

Las herramientas basadas en IA para generación de pruebas presentan, sin embargo, ventajas y limitaciones que conviene contrastar con la propuesta de este trabajo. *ChatUniTest* [24], *TestSpark* [25] y *Pynguin* [26] generan pruebas unitarias de forma eficaz, pero lo hacen a partir del código fuente preexistente, por lo que operan de forma natural en un esquema *test-last* y se integran en los entornos de desarrollo; *Pytester* [20] sí traduce descripciones textuales a casos

de prueba, aunque sin gestión estructurada de requisitos ni control del estado del ciclo. La ventaja de estas herramientas es su madurez y su cobertura, y su principal limitación, de cara al TDD, es que presuponen la existencia del código que la metodología precisamente propone escribir después de la prueba. La propuesta de este trabajo, denominada TDDMachine, se sitúa deliberadamente en una fase previa del ciclo: es una herramienta CASE (*Computer-Aided Software Engineering*) que interviene de manera exclusiva en la transición *Red*→*Green* del ciclo TDD, generando el script de prueba en Python con `pytest` a partir de especificaciones estructuradas, para entregar al equipo de desarrollo el punto de partida sobre el cual escribir el código de producción mínimo. La solución se desarrolló derivando sus requisitos de una revisión sistemática de la literatura, seleccionando su pila tecnológica (React 18, Django 4.2, PostgreSQL 15 y despliegue en contenedores Docker) mediante evaluación multicriterio ponderada, e integrando el modelo Claude 3.5 Sonnet a través de un constructor de *prompts* estructurados y un validador que verifica la sintaxis del código generado mediante el árbol sintáctico abstracto (AST) de Python. Para su comprobación se diseñaron dos estudios acordes con ese alcance: un estudio de usabilidad y aceptación tecnológica con 25 participantes, mediante los instrumentos SUS y TAM, y un estudio de calidad de las pruebas generadas con 10 expertos, mediante una rúbrica estructurada.

Las contribuciones de este trabajo son tres. Primera, TDDMachine, una herramienta CASE que integra en un único flujo la captura estructurada de tres tipos de especificación —requisitos funcionales, casos de uso e historias de usuario—, la generación del estado inicial de las pruebas mediante un LLM y la validación sintáctica del código generado mediante el AST de Python, brecha que ninguna de las herramientas del estado del arte cubre de forma integrada. Segunda, una revisión sistemática de 108 estudios primarios que caracteriza los estándares y notaciones para estructurar los requisitos y su relación con la generación automática de pruebas, y que fundamenta el diseño de la herramienta. Tercera, evidencia empírica de la viabilidad y la aceptación de la propuesta, obtenida mediante un estudio de usabilidad y aceptación tecnológica con 25 participantes y un estudio de calidad de las pruebas con 10 expertos.

El resto del artículo se organiza como sigue. La Sección II presenta una revisión sistemática de la literatura, conducida según los lineamientos de Kitchenham, que sitúa la contribución frente al estado del arte. La Sección III describe la metodología de desarrollo y evaluación. La Sección IV reporta los resultados de generación, usabilidad, aceptación y calidad. La Sección V discute los hallazgos frente a los objetivos y trabajos previos y expone las amenazas a la validez. Finalmente, la Sección VI resume las conclusiones y las líneas de trabajo futuro.

II. TRABAJOS RELACIONADOS

Con el fin de fundamentar los requisitos de TDDMachine y situar su aporte frente al estado del arte, se condujo una revisión sistemática de la literatura (*Systematic Literature Review*, SLR) siguiendo los lineamientos metodológicos de Kitchenham et al., que estructuran el proceso en tres fases —planificación, ejecución y reporte— [27], y se reportó conforme a la declaración PRISMA 2020 [28]. Antes de la ejecución se definió un protocolo que fija las preguntas de investigación, la estrategia y las cadenas de búsqueda, los criterios de inclusión y exclusión —que incorporan la valoración de calidad metodológica como criterio aplicado a texto completo (CE1)— y el procedimiento de extracción de datos, con el objeto de garantizar la trazabilidad y la reproducibilidad de la revisión.

A. Preguntas de investigación

El objetivo de la revisión es determinar cómo debe capturarse de forma estructurada la especificación de software para habilitar la generación automática de pruebas, cuestión que fundamenta el diseño de los formularios de entrada de TDDMachine. De este objetivo se

TABLE I
PREGUNTAS DE INVESTIGACIÓN DE LA REVISIÓN SISTEMÁTICA

ID	Pregunta de investigación
RQ1	¿Cuáles son los estándares y metodologías existentes para la captura estructurada de requisitos funcionales?
RQ2	¿Qué características deben tener los formularios y esquemas para estandarizar la captura de especificaciones?
RQ3	¿Cómo facilitan estos estándares la generación automática de casos de prueba?
RQ4	¿Qué herramientas CASE existentes implementan estas metodologías?

TABLE II
OPERACIONALIZACIÓN DE LA BÚSQUEDA MEDIANTE PICOC

Elemento	Términos
Población (P)	Sistemas de software, aplicaciones web y proyectos de desarrollo de software.
Intervención (I)	Estándares de captura de requisitos, formularios estructurados, metodologías de documentación y herramientas CASE.
Comparación (C)	Enfoques tradicionales frente a enfoques estructurados; distintos estándares de documentación.
Resultados (O)	Estandarización de la especificación, facilidad para la generación automática de pruebas y reducción del tiempo de documentación.
Contexto (C)	Ingeniería de requisitos, desarrollo de software y pruebas automatizadas.

derivaron las cuatro preguntas de investigación de la Tabla I: RQ1 indaga los estándares existentes para estructurar los requisitos; RQ2, los campos que deben incluir los formularios de captura; RQ3, el modo en que dicha estructuración facilita la generación de pruebas; y RQ4, las herramientas CASE que ya implementan estas ideas y sus limitaciones.

B. Estrategia y cadenas de búsqueda

La estrategia de búsqueda se estructuró con el marco PICOC (*Population, Intervention, Comparison, Outcomes, Context*), que operacionaliza el objetivo de la revisión en cinco elementos. La Tabla II detalla los términos asignados a cada elemento.

Estos cinco elementos delimitan la población de interés (proyectos y sistemas de software), la intervención (estándares, formularios y herramientas de captura estructurada de requisitos), la comparación (enfoques estructurados frente a los tradicionales), los resultados esperados (estandarización de la especificación y generación automática de pruebas) y el contexto (ingeniería de requisitos y pruebas automatizadas). A partir de estos términos se construyeron cadenas de búsqueda booleanas específicas para cada tipo de especificación —requisitos funcionales, casos de uso e historias de usuario—, que se aplicaron sobre cuatro bases de datos (Scopus, IEEE Xplore, ACM Digital Library y Web of Science) y se complementaron con la exploración de repositorios de código abierto (GitHub) para identificar implementaciones prácticas. La Tabla III presenta dichas cadenas en su forma canónica.

Como muestra la Tabla III, las cadenas emplean truncamiento (*) y sinónimos idiomáticos para maximizar la cobertura, y se adaptaron a la sintaxis de cada base —Scopus (TITLE-ABS-KEY), Web of Science (TS=), IEEE Xplore ("All Metadata") y ACM Digital Library

(AllField)— preservando los mismos bloques conceptuales, de modo que la recuperación resultara comparable entre las cuatro fuentes.

C. Criterios de selección y proceso de cribado

La selección de los estudios se rigió por los criterios de inclusión (CI) y exclusión (CE) de la Tabla IV, definidos a priori y asignados cada uno a una única etapa del proceso para evitar su aplicación duplicada, tal como indica la columna *Etapa* de dicha tabla.

El proceso, ilustrado en el diagrama PRISMA de la Fig. 1, comprende cuatro filtros consecutivos. El primero es la identificación de los registros recuperados por las cadenas de la Tabla III en cada base. El segundo elimina los duplicados (CE5), dando prioridad a las fuentes primarias (ACM Digital Library e IEEE Xplore) sobre los índices bibliográficos (Scopus y Web of Science), que replican gran parte de su contenido. El tercero es la elegibilidad, en la que se aplican de una sola vez, y sobre los metadatos de las bases, todos los criterios formales: año de publicación desde 2015 (CI1), idioma inglés (CI6) y tipo de documento admisible —estudios primarios: artículos de revista, ponencias de congreso, capítulos de libro, libros y tesis doctorales (CI7)—, con la exclusión de los no-artículos —actas y material preliminar de congresos— (CE4) y de los estudios secundarios —revisiones, *surveys* y mapeos sistemáticos— (CE3). El cuarto es el cribado por título y resumen, que retiene únicamente los estudios pertinentes al objetivo operativo de la revisión —que un artefacto de requisitos estructurado se vincule de forma explícita con la generación o derivación automática de pruebas (CI2–CI5; CE2)—. De este modo, el tipo de documento se resuelve una sola vez en la elegibilidad, el cribado por título y resumen queda circunscrito a la pertinencia temática, y la etapa final —la lectura a texto completo— se reserva a la evaluación de la calidad metodológica (CE1) y a la confirmación del acceso al documento (CE6).

La Fig. 1 cuantifica el resultado de cada filtro. Partiendo de 6795 registros identificados en las cuatro bases, la eliminación de duplicados —concentrada, como corresponde, en Scopus y Web of Science— deja 5628 registros únicos; los criterios de elegibilidad —año, idioma y tipo de documento— los reducen a 2642 (se descartan 2758 registros por año o idioma y 228 por tipo de documento); y el cribado por título y resumen —incorporando estudios complementarios para la categoría de historias de usuario y un estudio del volumen ICSR 2016— deja 154 estudios seleccionados a texto completo. Sobre estos 154 se realizó la lectura a texto completo, gobernada por el criterio de calidad metodológica (CE1): 108 estudios quedaron incluidos y 46 se excluyeron —16 por calidad metodológica insuficiente y 30 cuya condición de fuera del alcance temático (27) o de estudio secundario (3) solo pudo confirmarse con el documento completo—. La Tabla V desglosa la identificación, la deduplicación y la elegibilidad por categoría de especificación y base de datos.

El desglose de la Tabla V muestra que los requisitos funcionales concentran la mayor parte de los registros recuperados (5227 de 6795) y que la reducción por deduplicación y elegibilidad es proporcionalmente mayor en Scopus y Web of Science, coherente con su papel de índices que replican en buena medida la producción de las bibliotecas primarias. A partir de los 2642 registros elegibles, la Tabla VI reporta, por categoría, cuántos estudios se seleccionaron para la lectura a texto completo y cuántos quedaron finalmente incluidos.

Como recoge la Tabla VI, de los 154 estudios leídos a texto completo se incluyeron 108, con tasas de inclusión del 71 % en requisitos funcionales (86 de 121), del 59 % en casos de uso (13 de 22) y del 82 % en historias de usuario (9 de 11); esta última categoría, pese a su menor volumen absoluto, aportó la proporción más alta de estudios pertinentes al objetivo de la revisión.

D. Extracción de datos y síntesis por pregunta de investigación

De cada estudio incluido se extrajeron, mediante una plantilla estandarizada, los siguientes campos: año, autores, DOI, tipo de estudio,

TABLE III
CADENAS DE BÚSQUEDA EN FORMA CANÓNICA (ADAPTADAS A LA SINTAXIS DE CADA BASE DE DATOS)

Categoría	Cadena de búsqueda
Requisitos funcionales	("requirement* specification" OR "requirement* template" OR "requirement* boilerplate" OR "requirement* pattern" OR "structured requirement*" OR "controlled natural language") AND ("requirements engineering" OR "software engineering")
Casos de uso	("use case template" OR "use case specification" OR "use case model*" OR "use case description" OR "restricted use case" OR "structured use case") AND ("requirements engineering" OR "software engineering")
Historias de usuario	("user stor* template" OR "user stor* format" OR "user stor* pattern" OR "user stor* documentation" OR "structured user stor*") AND ("acceptance criteria" OR "INVEST" OR "agile" OR "scrum" OR "requirements engineering")

TABLE IV
CRITERIOS DE INCLUSIÓN Y EXCLUSIÓN APLICADOS

Crterios de inclusión (CI)		Etap
CI1	Publicados entre 2015 y la fecha de ejecución de la búsqueda.	Elegib.
CI2	Describen estándares, modelos o metodologías de captura estructurada de requisitos.	Tít./res.
CI3	Presentan formularios, esquemas o criterios para estandarizar especificaciones.	Tít./res.
CI4	Abordan la relación entre estructuración de requisitos y generación de pruebas.	Tít./res.
CI5	Tratan herramientas CASE para la gestión de requisitos.	Tít./res.
CI6	Publicados en inglés.	Elegib.
CI7	Estudios primarios: artículos de revista, ponencias de congreso, capítulos de libro, libros o tesis doctorales.	Elegib.
Crterios de exclusión (CE)		Etap
CE1	Estudios puramente teóricos sin aplicación práctica, o que no presentan evidencia empírica, rigor metodológico ni claridad de reporte suficientes (calidad insuficiente).	Texto compl.
CE2	Fuera del alcance temático: no abordan la captura estructurada de requisitos ni su relación con la generación de pruebas.	Tít./res.
CE3	Estudios secundarios (revisiones sistemáticas, <i>surveys</i> y mapeos), que no aportan evidencia primaria.	Elegib.
CE4	No-artículos (actas, prefacios y material preliminar de congresos) y literatura gris sin revisión por pares (salvo estándares oficiales).	Elegib.
CE5	Duplicados, versiones preliminares o solapamientos entre categorías.	Deduplic.
CE6	Sin acceso al texto completo.	Texto compl.

La columna *Etap* indica el único filtro del flujo PRISMA (Fig. 1) en el que se aplica cada criterio; Deduplic. = deduplicación; Elegib. = elegibilidad (metadatos: año, idioma y tipo de documento); Tít./res. = cribado por título y resumen (pertinencia temática); Texto compl. = lectura a texto completo (calidad metodológica).

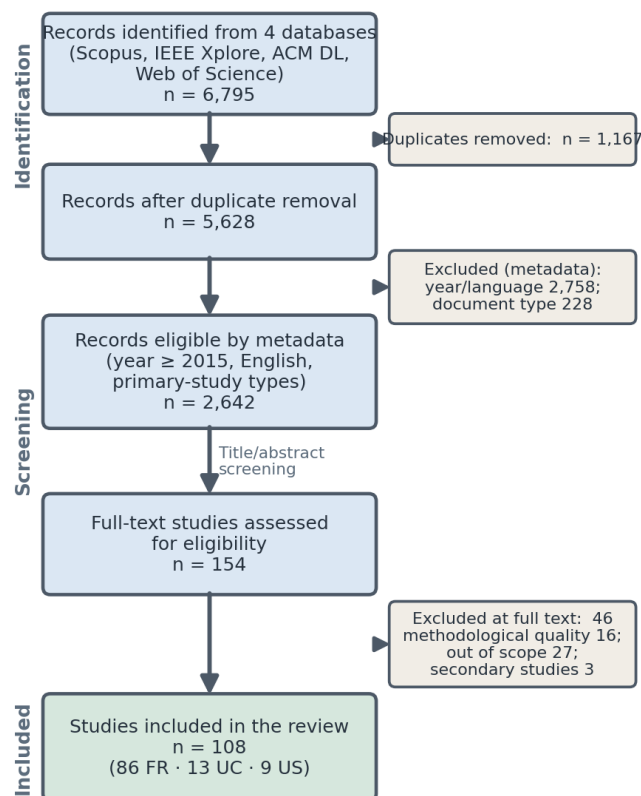


Fig. 1. Flujo PRISMA consolidado de la revisión sistemática (requisitos funcionales, casos de uso e historias de usuario en las cuatro bases). La elegibilidad aplica en una sola etapa los criterios de año, idioma y tipo de documento; el cribado por título y resumen se circunscribe a la pertinencia temática; y de los 154 estudios leídos a texto completo, 108 resultaron incluidos tras valorar la calidad metodológica.

contexto de aplicación, estándares o metodologías descritos, campos de especificación identificados, herramientas CASE mencionadas y relación con la generación automática de pruebas. La extracción completa de los 108 estudios incluidos —distribuidos en 86 de requisitos funcionales, 13 de casos de uso y 9 de historias de usuario, y compuesta por 72 ponencias de congreso, 35 artículos de revista y una tesis doctoral— se proporciona como material complementario y el conjunto se referencia de forma exhaustiva en [29]–[136]; la Fig. 2 presenta la distribución temporal de este conjunto por categoría de

TABLE V
IDENTIFICACIÓN, DEDUPLICACIÓN Y ELEGIBILIDAD POR CATEGORÍA DE ESPECIFICACIÓN Y BASE DE DATOS

Especificación	Base de datos	Identificados	Tras duplicados	Tras elegibilidad ^b
Requisitos funcionales	Scopus	1 646	1 312	522
	Web of Science	1 607	1 066	379
	IEEE Xplore	961	947	409
	ACM Digital Library	1 013 ^a	1 013	859
	Subtotal	5 227	4 338	2 169
Casos de uso	Scopus	483	360	116
	Web of Science	310	173	55
	IEEE Xplore	138	136	49
	ACM Digital Library	539	539	191
	Subtotal	1 470	1 208	411
Historias de usuario	Scopus	25	20	18
	Web of Science	18	7	4
	IEEE Xplore	6	6	4
	ACM Digital Library	49	49	36
	Subtotal	98	82	62
Total		6 795	5 628	2 642

Los duplicados se eliminaron dando prioridad a las fuentes primarias (ACM, IEEE) sobre los índices (Scopus, Web of Science). ^a La exportación de ACM para requisitos funcionales se acotó a 2015–2026 (por el límite de exportación de la plataforma), por lo que su columna de identificados ya excluye lo anterior a 2015.

^b Elegibilidad: se aplican en una sola etapa todos los criterios formales sobre los metadatos —año de publicación ≥ 2015 (CI1), idioma inglés (CI6) y tipo de documento admisible: estudios primarios (artículos de revista, ponencias de congreso, capítulos de libro, libros y tesis doctorales; CI7), excluyendo no-artículos (CE4) y estudios secundarios (CE3)—. De los 5 628 registros únicos, 2 758 se descartaron por año o idioma y 228 por tipo de documento; sobre los 2 642 elegibles se aplicó el cribado por título y resumen (Tabla VI).

TABLE VI
ESTUDIOS SELECCIONADOS A TEXTO COMPLETO E INCLUIDOS, POR CATEGORÍA

Categoría	Seleccionados a texto completo ^a	Incluidos ^b
Requisitos funcionales	121	86
Casos de uso	22	13
Historias de usuario	11	9
Total	154	108

^a Registros que superan el cribado por título y resumen —circunscrito a la pertinencia temática con el objetivo de la revisión (CI2–CI5; CE2)—, partiendo de los 2 642 registros ya elegibles por año, idioma y tipo de documento; incorpora estudios complementarios para la categoría de historias de usuario y un estudio del volumen ICSR 2016. ^b Estudios que, tras la lectura a texto completo, cumplen la calidad metodológica suficiente (CE1). De los 154 leídos, 46 se excluyeron: 16 por calidad metodológica insuficiente y 30 cuya condición de fuera del alcance temático (27) o de estudio secundario (3) solo pudo confirmarse con el documento completo.

artefacto de requisitos.

Como se observa en la Fig. 2, la producción sobre requisitos funcionales se mantiene sostenida a lo largo de toda la ventana temporal (2015–2026) y concentra la mayor parte del corpus, mientras que el incremento observado en los años más recientes coincide con la difusión de los modelos de lenguaje de gran escala en la ingeniería de software. Sobre este mismo conjunto de estudios, la Tabla VII sintetiza las frecuencias de estándares, notaciones y técnicas de derivación de pruebas observadas en el texto completo, cuyo análisis se organiza a

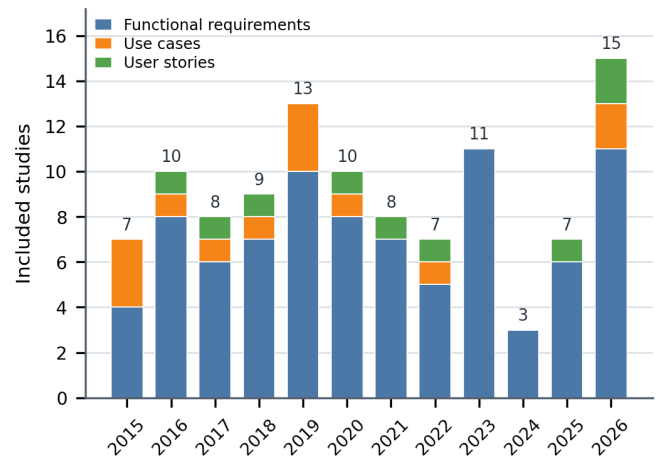


Fig. 2. Distribución temporal de los 108 estudios incluidos (2015–2026), apilada por categoría de artefacto de requisitos. La cifra sobre cada barra indica el total del año.

continuación en torno a las cuatro preguntas de investigación.

RQ1 (estándares para la captura estructurada). La evidencia muestra que ningún estándar normativo único gobierna la práctica; en cambio, cada tipo de especificación se estructura mediante una notación predominante. Los casos de uso se capturan mayoritariamente con modelos UML —diagramas de caso de uso, de secuencia o máquinas de estado, presentes en el 68 % de los estudios [31]–[103]— y con plantillas de estilo Cockburn o su formalización RUCM (20 %) [86]–

TABLE VII

SÍNTESIS DE LA EVIDENCIA DE LOS 108 ESTUDIOS INCLUIDOS:
FRECUENCIA DE ESTÁNDARES/NOTACIONES Y DE TÉCNICAS DE
DERIVACIÓN DE PRUEBAS (PRESENCIA EN EL TEXTO COMPLETO)

Estándar / notación de especificación	%
Modelos UML (diagramas de caso de uso, secuencia, máquina de estados)	68
Narrativa de historia de usuario (rol–objetivo, 3C)	36
Gherkin / BDD (<i>Given–When–Then</i>)	33
TDD / ATDD (criterios de aceptación ejecutables)	25
Lenguaje natural controlado (CNL)	21
Plantillas de casos de uso (Cockburn / RUCM)	20
Criterios de aceptación	21
Ontologías de requisitos	17
EARS (requisitos funcionales)	9
INVEST (historias de usuario)	6
ISO/IEC/IEEE 29148 (marco normativo)	5
Técnica de derivación de pruebas	%
Formalización / plantillas / gramáticas	77
Procesamiento de lenguaje natural (NLP)	44
Pruebas basadas en modelos (MBT)	42
Modelos de lenguaje de gran escala (LLM)	24
Ontologías / web semántica	17

Frecuencias calculadas sobre los 108 estudios incluidos. Para los 107 estudios con texto completo legible por máquina, la presencia de cada estándar, notación o técnica se determinó mediante análisis léxico automático; para la tesis doctoral restante —sin texto procesable por dicho análisis— se determinó mediante codificación manual de su texto completo. Las categorías no son mutuamente excluyentes.

[107]; los requisitos funcionales, con lenguaje natural controlado (21 %) [71]–[87], [102]–[105], [120], [121] y con el enfoque EARS (*Easy Approach to Requirements Syntax*, 9 %) [42]–[45], [82]–[86], [136]; y las historias de usuario, con la narrativa estructurada rol–objetivo y el esquema 3C (36 %) [33]–[42], [75]–[82], [97]–[102], [106]–[120], el modelo INVEST (6 %) [36], [37], [49], [50], [111], [122] y los escenarios Gherkin/BDD (33 %) [29]–[36], [45]–[49], [73]–[75], [79]–[82], [86]–[89], [99]–[102], [106], [114]–[120]. El marco normativo ISO/IEC/IEEE 29148:2018 [9] actúa como paraguas de referencia, aunque se cita explícitamente en una minoría de estudios (5 %) [34], [39], [101], [107], [108]. Los estudios restantes no adoptan una notación canónica de estructuración: operan sobre especificaciones formales ([126]–[129]), lenguajes específicos de dominio o marcos genéricos ([123]–[125]), o lenguaje natural no controlado procesado mediante NLP/LLM ([130]–[135]). Esta distribución fundamenta que TDDMachine soporte los tres artefactos con sus respectivas estructuras predominantes.

RQ2 (características de los formularios). Del análisis de las plantillas descritas por los estudios incluidos se identificaron los campos recurrentes que todo formulario debe capturar: para los requisitos funcionales, nombre, descripción, tipo, criterios de aceptación y prioridad; para los casos de uso, nombre, actores, objetivo, el flujo básico y los flujos alternativos [58], [68], [71], [76], [86]–[94], [96], [97], [101]–[104], [119], [120] y las precondiciones y postcondiciones [31]–[33], [36], [39], [42], [48], [49], [54], [55], [66], [67], [69], [70], [75], [79], [83], [84], [86]–[97], [101]–[103], [106], [107], [109], [115], [116], [119], [125], [126], [128], [129], [132]; y para las historias de usuario, la narrativa estructurada “como [rol], quiero [objetivo], para [beneficio]” [33]–[42], [75]–[82], [97]–[102], [106]–[120] junto con sus criterios de aceptación [32], [33], [38]–[40],

[79]–[82], [84], [86], [97], [99], [101], [106]–[108], [112], [113], [115], [116], [120]. Estos campos, coincidentes con las plantillas canónicas de Cockburn y su formalización RUCM [86]–[107], con el estándar ISO/IEC/IEEE 29148 y con el modelo INVEST/Connextra, se implementaron como obligatorios en la herramienta; los de aparición minoritaria (estado, origen, dependencias o estimación) se dispusieron como opcionales.

RQ3 (relación con la generación de pruebas). Predominan las técnicas que explotan una entrada estructurada o formalizada: la formalización mediante plantillas, gramáticas o lenguaje natural controlado aparece en el 77 % de los estudios [30], [31], [33], [35]–[37], [39]–[43], [45]–[51], [53], [54], [56], [58]–[61], [64]–[66], [69]–[107], [109]–[112], [114], [117], [119]–[122], [124], [127], [129], [131], [132], [134]; el procesamiento de lenguaje natural, en el 44 % [30], [33], [35], [42], [44], [45], [58], [63], [65], [66], [69], [73], [74], [76]–[81], [83]–[92], [94], [96], [98], [101], [103], [105], [110], [111], [113], [120], [121], [124], [129]–[134]; las pruebas basadas en modelos, en el 42 % [29], [32], [33], [35], [36], [42], [45], [48]–[53], [56], [60], [64], [66], [67], [69]–[72], [74], [77]–[79], [82], [84], [85], [87]–[89], [92]–[97], [99], [102], [103], [116], [120], [124], [127]; y el empleo de modelos de lenguaje de gran escala, en el 24 % [39], [44], [51], [55], [61], [63], [66], [73], [78], [91], [94], [101], [108], [110]–[112], [120], [122], [124], [129]–[131], [133]–[136], con una tendencia creciente en los años más recientes. Los campos estructurados —en particular los criterios de aceptación y las pre/postcondiciones— resultan directamente transformables en aserciones verificables, de modo que la estandarización de la entrada reduce la ambigüedad del lenguaje natural y mejora la coherencia de las pruebas generadas automáticamente, en línea con la evidencia de que la calidad de la especificación de entrada determina la del resultado [2], [16].

RQ4 (herramientas CASE existentes). Las herramientas identificadas se agrupan en dos familias. La primera deriva pruebas a partir de especificaciones estructuradas de requisitos —por ejemplo, generadores desde casos de uso restringidos (RUCM/UMTG, aToucan, MARITACA), desde requisitos en lenguaje natural (CiRA, TestMEReq, EARS2TF) y, en los trabajos más recientes, asistidos por LLM (Req2Test, REST-at) [39], [44], [51], [55], [61], [63], [66], [73], [78], [91], [94], [101], [108], [110]–[112], [120], [122], [124], [129]–[131], [133]–[136]—, pero suele detenerse en la especificación de la prueba sin cubrir el ciclo TDD. La segunda genera pruebas a partir del código fuente o de texto libre —*ChatUniTest* [24], *TestSpark* [25], *Penguin* [26] y *Pytester* [20]—, por lo que opera en una fase posterior, en un esquema *test-last*. Ninguna integra la captura estructurada de requisitos con la generación de la prueba de la etapa *Red* asistida por un LLM, brecha que TDDMachine aborda. Del análisis se desprende, además, que los modelos de propósito general con elevada capacidad de síntesis y procesamiento de código —Claude 3.5 Sonnet y GPT-4— fueron los preferidos en la literatura reciente, en coherencia con la revisión de LLM para ingeniería de software de Hou et al. [11].

E. Posicionamiento frente al estado del arte

Para situar la contribución no frente a un puñado de sistemas sino frente a un conjunto representativo del estado del arte, la Tabla VIII contrasta TDDMachine con catorce herramientas representativas del estado del arte, agrupadas en tres familias: (i) generadores de pruebas a partir de especificaciones estructuradas de requisitos —casos de uso restringidos con UMTG [96], RTCM/aToucan [102], MARITACA [92] y la generación de pruebas de aceptación desde casos de uso [87]; requisitos en lenguaje natural con CiRA [113], TestMEReq [109], EARS2TF [85] y la conversión de requisitos a casos de prueba [38]—; (ii) propuestas recientes asistidas por LLM —Req2Test [120] y REST-at [131]—; y (iii) generadores que parten del código fuente o de texto libre —*ChatUniTest* [24], *TestSpark* [25], *Penguin* [26] y *Pytester* [20]. El contraste evidencia que las herramientas basadas en requisitos rara vez integran un motor LLM y ninguna incorpora validación del

TABLE VIII
COMPARACIÓN DE TDDMACHINE CON CATORCE HERRAMIENTAS REPRESENTATIVAS DEL ESTADO DEL ARTE

Herramienta	Entrada ^a	Artefacto	Motor ^b	Valid. AST	Gestión ^c
TDDMachine (este trabajo)	RE	RF, CU, HU	LLM	Sí	Sí
<i>Generadores desde especificaciones estructuradas de requisitos</i>					
UMTG [96]	RE	CU	NLP	No	Parc.
RTCM / aToucan [102]	RE	CU	NLP	No	No
Pruebas de aceptación desde CU [87]	RE	CU	NLP	No	No
MARITACA [92]	RE	CU	NLP/MBT	No	No
CiRA [113]	LN	RF	NLP/ML	No	Parc.
TestMEReq [109]	RE	RF	MBT	No	No
EARS2TF [85]	RE	RF	Semiformal	No	No
Convertir requisitos a pruebas [38]	RE	RF	NLP	No	No
<i>Propuestas asistidas por LLM</i>					
Req2Test [120]	RE	HU	LLM	No	Parc.
REST-at [131]	RE	RF	LLM	No	Parc.
<i>Generadores desde código fuente o texto libre (test-last)</i>					
ChatUniTest [24]	CF	—	LLM	Parc.	Parc.
TestSpark [25]	CF	—	LLM/EV	Parc.	Sí
Pynguin [26]	CF	—	EV	Sí	No
Pytester [20]	TL	—	RL	No	No

^a Entrada: RE=requisitos estructurados; LN=requisitos en lenguaje natural; CF=código fuente; TL=texto libre. ^b Motor: LLM=modelo de lenguaje de gran escala; NLP=procesamiento de lenguaje natural; MBT=pruebas basadas en modelos; EV=algoritmos evolutivos; RL=aprendizaje por refuerzo; ML=aprendizaje automático. ^c Gestión: capacidades integradas de revisión, edición y exportación del ciclo de pruebas. Únicamente TDDMachine interviene de forma explícita en la etapa *Red* del ciclo TDD (antes de que exista código de producción).

código generado mediante AST ni la gestión y exportación del ciclo de pruebas; las basadas en LLM no capturan de forma estructurada los tres tipos de artefacto; y las orientadas al código operan en un esquema *test-last*, una vez que existe el código de producción. TDDMachine es la única propuesta que reúne captura estructurada de los tres artefactos, motor LLM, validación sintáctica por AST y gestión integrada, situándose además de forma explícita en la etapa *Red* del ciclo TDD.

F. Amenazas a la validez de la revisión

La revisión presenta las limitaciones habituales de una SLR acotada. La validez de constructo está condicionada por la restricción a cuatro bases de datos (Scopus, IEEE Xplore, ACM Digital Library y Web of Science) y a publicaciones en inglés desde 2015, lo que pudo dejar fuera literatura relevante en otras fuentes o idiomas; se mitigó mediante cadenas de búsqueda construidas con el enfoque PICOC —con truncamiento y sinónimos para maximizar la cobertura— y complementadas con la exploración de repositorios de código abierto. Para acotar el riesgo de sesgo de selección, el cribado y la extracción se condujeron mediante un protocolo de revisión por múltiples autores: el segundo autor ejecutó la selección inicial de los estudios; el primer autor revisó el proceso paso a paso para corroborar y corregir las decisiones; y los autores tercero, cuarto y quinto realizaron una revisión completa del proceso. Las discrepancias se resolvieron por consenso y, en su defecto, por mayoría entre los cinco autores. Este procedimiento, junto con la aplicación de criterios de inclusión y exclusión explícitos, una plantilla de extracción estandarizada y el registro documentado de las razones de exclusión, mitiga el sesgo de un revisor único y fortalece la fiabilidad de la selección. La validez interna conserva, no obstante, la limitación inherente a la interpretación humana de los criterios, atenuada por la triangulación entre revisores.

III. METODOLOGÍA

El desarrollo de TDDMachine se organizó en seis fases sucesivas —análisis de requisitos, especificación, diseño arquitectónico, implementación de módulos, verificación y validación, y despliegue—, alineadas con los procesos técnicos del ciclo de vida del software de la norma ISO/IEC/IEEE 12207:2017 [137] y conducidas bajo un enfoque de ingeniería de software basada en componentes (*Component-Based Software Engineering*, CBSE); la Sección III-A recorre cada una de ellas y la Sección III-B desarrolla la fase de verificación y validación. El alcance de la herramienta se delimitó de forma explícita: interviene únicamente en la transición *Red*→*Green* del ciclo TDD, generando el script de prueba que el equipo de desarrollo utilizará como estado inicial, mientras que las etapas *Green* —la escritura del código de producción— y *Refactor* quedan fuera de su alcance. Puesto que la herramienta materializa ese estado inicial en un artefacto de prueba ejecutable, debe fijar un lenguaje y un marco de pruebas objetivo: se adoptaron Python y *pytest*, seleccionados mediante la evaluación multicriterio descrita más adelante y porque el árbol sintáctico abstracto (AST) nativo de Python habilita la validación sintáctica automática que caracteriza a la propuesta.

A. Diseño y desarrollo de la herramienta

En la fase de *análisis de requisitos*, estos se derivaron de la revisión sistemática descrita en la Sección II: a partir de los campos identificados en las plantillas de los estudios incluidos y en los estándares de referencia se determinó qué información capturar en cada formulario, clasificando como obligatorios los campos recurrentes —los presentes de forma consistente en dichas plantillas— y como opcionales los restantes. De este análisis se obtuvieron dieciocho requisitos funcionales organizados en cuatro módulos —autenticación y gestión de usuarios, gestión de proyectos, captura de especificaciones, y generación y gestión de pruebas— y un conjunto de requisitos

no funcionales de seguridad, rendimiento, usabilidad, integración y disponibilidad, especificados conforme a ISO/IEC/IEEE 29148:2018 e ISO/IEC 25010 [9]. Como parte de la misma fase de *especificación*, la selección de los componentes tecnológicos se realizó mediante una matriz de decisión multicriterio ponderada, en la que cada alternativa se calificó de 1 a 5 sobre criterios técnicos con pesos porcentuales; resultaron seleccionados React 18 con Ant Design en el *frontend*, Django 4.2 con Django REST Framework en el *backend*, PostgreSQL 15 como gestor de base de datos y Claude 3.5 Sonnet como modelo de generación.

El *diseño arquitectónico* definió, sobre esos requisitos, una arquitectura cliente-servidor con separación estricta de capas, resumida en la Fig. 3. El *frontend* en React 18 se comunica con el *backend* Django mediante una API REST, con interceptores que adjuntan el token JWT a cada solicitud. La capa de lógica de negocio implementa tres servicios principales que materializan el flujo de generación: *PromptBuilder*, que construye *prompts* estructurados a partir del tipo y el contenido de cada especificación; *ClaudeService*, que gestiona las llamadas a la API de Anthropic con reintentos automáticos y espera exponencial ante fallos transitorios; y *PytestValidator*, que verifica la sintaxis del código generado analizando el árbol sintáctico abstracto (AST) de Python. Esta separación aísla la construcción del *prompt*, la invocación del modelo y el control de calidad sintáctico, de modo que cada etapa puede evolucionar o sustituirse de forma independiente.

Ya en la *implementación de módulos*, y en cuanto a la estructura del script generado, el servicio *PromptBuilder* instruye al modelo para que produzca pruebas *pytest* organizadas con *fixtures* —el mecanismo de *pytest* para la preparación y el desmontaje del entorno de prueba (*setup/teardown*)— y aserciones derivadas de los criterios de aceptación de la especificación. Un aspecto propio de la etapa *Red* es el tratamiento de las dependencias externas del código aún inexistente, en particular el acceso a la base de datos, la invocación del LLM o la comunicación de red: el estado inicial de la prueba debe aislar la unidad bajo prueba de esas dependencias mediante *dobles de prueba* (*test doubles*). En el ecosistema de Python este aislamiento se realiza con la biblioteca estándar *unittest.mock* —que permite crear objetos simulados (*mocks*) y sustituir dependencias mediante *patch*, de forma análoga a *Mockito* en Java— y su envoltorio para *pytest*, el complemento *pytest-mock* (*fixture mocker*); para las pruebas que sí deben ejercitar la capa de persistencia, el complemento *pytest-django* aporta una base de datos de prueba transaccional a través de la *fixture db*. Por ello, las directrices del *PromptBuilder* contemplan la generación de *fixtures* y, cuando la especificación involucra acceso a datos, la simulación de dichas dependencias; el refuerzo de la generación automática de *mocks* y *fixtures* para dependencias de datos complejas constituye una de las líneas de mejora identificadas en la evaluación experta.

El resultado de la validación determina el tratamiento de cada prueba: cuando es exitosa, se almacena con estado “Generada”; ante un error de sintaxis, se registra con estado “Error de generación” y se presenta igualmente al usuario para su corrección manual en un editor Monaco integrado con resaltado de sintaxis, autocompletado y detección de errores en tiempo real. El módulo de gestión permite revisar, clasificar (Generada, Revisada, Aprobada, Descartada) y exportar las pruebas aprobadas como un archivo *.py* descargable. La *verificación y validación* de la herramienta —mediante dos estudios empíricos— se aborda en la Sección III-B. Por último, el *despliegue* se realizó sobre infraestructura en la nube —*frontend* en Render, *backend* en contenedor Docker sobre Cloud Run y base de datos PostgreSQL gestionada en Supabase, con integración continua desde el repositorio GitHub—, lo que da continuidad a la línea de investigación del grupo sobre metodologías y herramientas guiadas por pruebas [138]–[140].

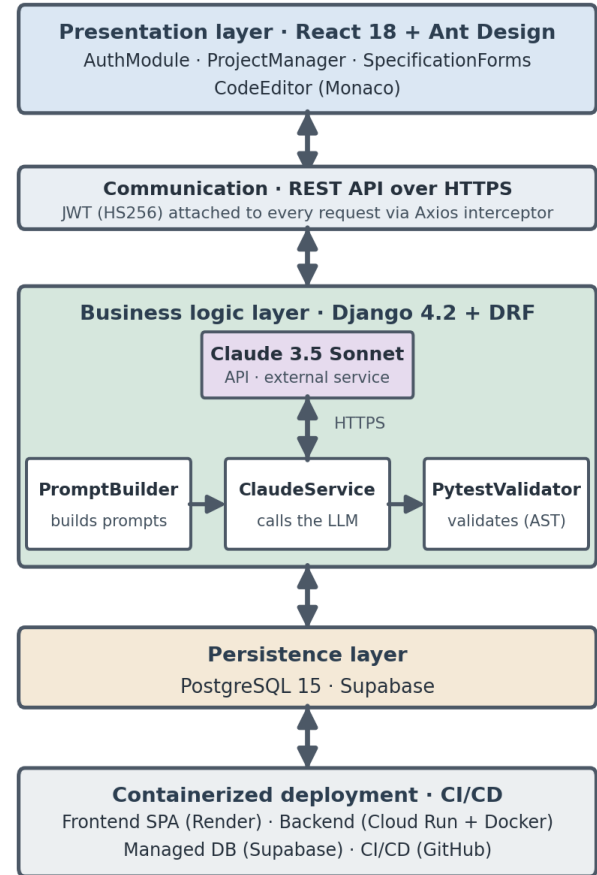


Fig. 3. Arquitectura en capas de TDDMachine. La capa de lógica de negocio encadena los servicios *PromptBuilder*, *ClaudeService* —que invoca el modelo Claude 3.5 Sonnet a través de la API de Anthropic— y *PytestValidator*, que valida el código generado mediante el árbol sintáctico abstracto (AST) de Python. Cada capa expone interfaces bien definidas, conforme al enfoque de ingeniería de software basada en componentes.

B. Diseño de la evaluación

La comprobación se estructuró en dos estudios complementarios, diseñados en concordancia con el alcance de la herramienta. El Estudio 1, de usabilidad y aceptación tecnológica, contó con la participación voluntaria de 25 estudiantes de sexto semestre de Ingeniería en Software, quienes completaron cuatro tareas que cubrieron el flujo de la etapa *Red*: creación de proyecto y registro de un caso de uso, especificación de un requisito funcional, redacción de una historia de usuario, y generación, revisión y exportación del script *pytest*. La usabilidad se midió con la escala SUS (*System Usability Scale*), cuestionario estandarizado de 10 ítems en escala Likert de 5 puntos cuyo puntaje se transforma a una escala de 0 a 100, interpretada según la escala de adjetivos de Bangor et al. [141], [142]. La aceptación tecnológica se midió con un instrumento basado en el modelo TAM (*Technology Acceptance Model*) de 16 ítems distribuidos en cuatro constructos —Utilidad Percibida (PU), Facilidad de Uso Percibida (PEOU), Confianza en Resultados (CR) e Intención de Uso (IU)—, con un umbral de aceptación de 3,5/5 por constructo [143].

El Estudio 2, de calidad de las pruebas, contó con 10 expertos en *testing* que evaluaron mediante una rúbrica estructurada, en escala de 1 a 5, cinco criterios —completitud, claridad, trazabilidad, viabilidad de ejecución y cobertura de escenarios— sobre 15 casos

TABLE IX
GENERACIÓN DE SCRIPTS `pytest` POR TIPO DE ESPECIFICACIÓN

Especificación	Intentos	Válidos	Error	Éxito
Requisitos funcionales	28	27	1	96,4 %
Casos de uso	24	22	2	91,7 %
Historias de usuario	18	15	3	83,3 %
Total	70	64	6	91,4 %

TABLE X
ESTADÍSTICA DESCRIPTIVA Y FIABILIDAD POR CONSTRUCTO TAM (N = 25)

Constructo	Ítems	Media	DE	α	Interp.
Utilidad Percibida (PU)	5	4,14	0,56	0,847	Alta
Facilidad de Uso (PEOU)	4	4,02	0,63	0,812	Alta
Confianza Result. (CR)	4	3,76	0,71	0,798	Mod.-Alta
Intención de Uso (IU)	3	4,08	0,60	0,831	Alta
Global	16	4,00	0,63	0,883	Alta

de prueba generados. La evaluación se realizó por inspección directa del código, sin requerir su ejecución, en coherencia con el alcance de la herramienta, que produce el estado inicial de la prueba y no el código de producción que la hace pasar.

IV. RESULTADOS

A. Efectividad de la generación automática

Durante la fase de evaluación se registraron 70 intentos de generación de scripts `pytest` a partir de las especificaciones capturadas. La Tabla IX resume los resultados por tipo de especificación: la tasa global de scripts válidos fue del 91,4 %, con el mejor desempeño en requisitos funcionales (96,4 %) y el menor en historias de usuario (83,3 %). Los seis intentos fallidos correspondieron a errores detectados por `PytestValidator` durante el análisis del AST —en cuatro casos, bloques de código incompletos atribuibles a especificaciones con criterios de aceptación ambiguos, y en dos, errores estructurales en la función de prueba—; en todos ellos el sistema preservó la trazabilidad registrando la prueba con estado “Error de generación” para su corrección manual.

B. Usabilidad y aceptación tecnológica

Los 25 participantes completaron las cuatro tareas en un tiempo medio de 42,0 minutos (DE=8,2). El puntaje SUS promedio fue de 74,46/100 (DE=6,55; mínimo 62,5; máximo 85,0), que supera el umbral de “Bueno” (68 puntos) de la escala de Bangor et al. y sitúa a la herramienta por encima del promedio de los sistemas evaluados con este instrumento [142]. En cuanto a la aceptación tecnológica, la Tabla X presenta la estadística descriptiva y la fiabilidad por constructo del modelo TAM.

Como muestra la Tabla X, los cuatro constructos superaron el umbral de 3,5/5: la Utilidad Percibida (4,14) y la Intención de Uso (4,08) obtuvieron los valores más altos, mientras que la Confianza en Resultados (3,76) fue el constructo más próximo al umbral, lo que refleja la incertidumbre de los participantes respecto de la fiabilidad del código generado por IA. El instrumento presentó una consistencia interna adecuada, con un coeficiente Alfa de Cronbach global de 0,883 y valores por constructo superiores a 0,79. La evidencia a nivel de participante se sintetiza en la Fig. 4, que reúne la distribución de los puntajes SUS individuales (a) y la matriz de correlaciones de Pearson entre los constructos TAM (b).

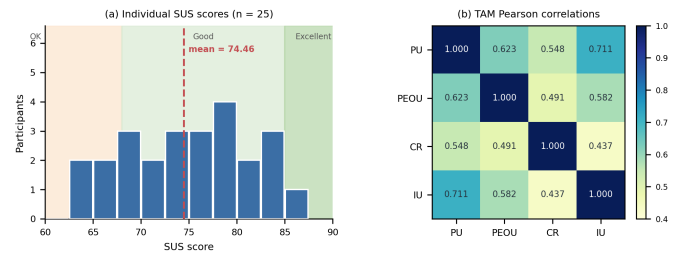


Fig. 4. Resultados del estudio de usabilidad y aceptación tecnológica (n=25). (a) Distribución de los puntajes SUS individuales sobre las bandas interpretativas de la escala de Bangor et al.; la línea discontinua indica la media (74,46). (b) Matriz de correlaciones de Pearson entre los cuatro constructos del modelo TAM; la intensidad del color representa la magnitud del coeficiente.

TABLE XI
CALIDAD DE LAS PRUEBAS POR CRITERIO (ESCALA 1–5, N = 10 EXPERTOS)

Criterio	Media	DE	Interpretación
Complejidad	4,2	0,45	Muy bueno
Claridad	4,3	0,42	Muy bueno
Trazabilidad	4,0	0,58	Bueno
Viabilidad de ejecución	3,9	0,67	Bueno
Cobertura de escenarios	4,0	0,61	Bueno
Media global	4,08	0,55	Muy bueno

La distribución de la Fig. 4(a) indica que el 80 % de los participantes (20 de 25) alcanzó al menos el umbral de “Bueno” (SUS $\geq$ 68), sin ningún caso por debajo del nivel mínimo aceptable de la escala. El mapa de correlaciones de la Fig. 4(b), por su parte, muestra que todas las asociaciones entre constructos son positivas y estadísticamente significativas ($p < 0,05$) y que la más fuerte se produce entre la Utilidad Percibida y la Intención de Uso ($r = 0,711$; $p < 0,01$), lo que confirma el supuesto central del modelo TAM.

C. Calidad de las pruebas generadas

La evaluación experta arrojó una calidad media global de 4,08/5,0 sobre los cinco criterios de la rúbrica. La Tabla XI presenta la valoración media, la desviación estándar y la interpretación por criterio.

Según la Tabla XI, los criterios mejor valorados fueron la claridad (4,3) y la completitud (4,2), calificados como “Muy bueno”, mientras que la viabilidad de ejecución (3,9) resultó el criterio más próximo al umbral, lo que sugiere oportunidades de mejora en la precisión de las aserciones respecto de las condiciones de la especificación de origen. Al desagregar por tipo de prueba, las pruebas unitarias generadas desde requisitos funcionales alcanzaron la calidad más alta (4,24), seguidas de las de integración desde casos de uso (4,08) y las de sistema desde historias de usuario (3,92). La Fig. 5 relaciona esta desagregación con la validez de generación de la Tabla IX según el tipo de artefacto de requisitos de origen.

Como evidencia la Fig. 5, ambos indicadores exhiben una tendencia concordante: al pasar de los requisitos funcionales a los casos de uso y a las historias de usuario —es decir, a medida que disminuye la estructura de la especificación y aumenta su ambigüedad—, la proporción de scripts válidos desciende del 96,4 % al 91,7 % y al 83,3 %, y la calidad experta lo hace de 4,24 a 4,08 y a 3,92. Esta concordancia sugiere que el grado de estructura del artefacto de requisitos constituye un determinante común del éxito de la generación y de la calidad de las pruebas resultantes.

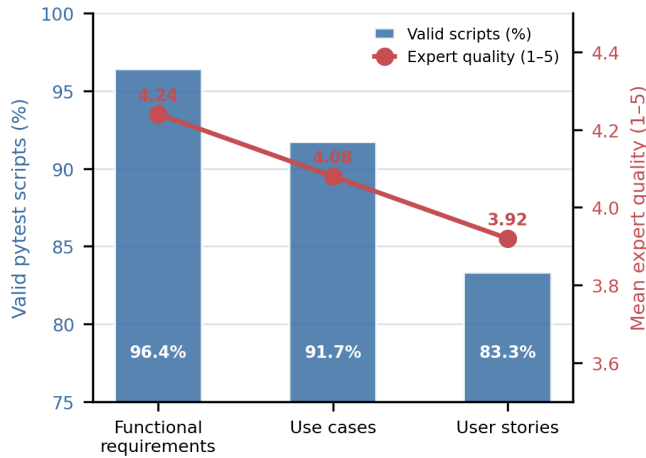


Fig. 5. Validez de generación de scripts `pytest` (Tabla IX) y calidad experta media de las pruebas (Tabla XI) según el tipo de artefacto de requisitos de origen. Ambos indicadores descienden de forma concordante desde los requisitos funcionales hacia las historias de usuario, conforme aumenta la ambigüedad de la especificación.

V. DISCUSIÓN

Los hallazgos de este trabajo se interpretan a la luz del cuerpo de 108 estudios primarios incluidos en la revisión sistemática [29]–[136], que delimita el estado del arte en la captura estructurada de requisitos y su relación con la generación automática de pruebas. Los resultados aportan evidencia de que la generación estructurada del estado inicial de las pruebas asistida por un LLM es viable y bien aceptada. La tasa global de scripts válidos del 91,4 % es superior al 78 % reportado por *Pytester* para la generación de casos de prueba ejecutables desde texto en lenguaje natural [20], lo que sugiere que la combinación de captura estructurada de especificaciones y validación sintáctica automática mediante AST constituye una mejora metodológica concreta respecto de enfoques que operan sobre texto libre sin verificación integrada. Este hallazgo es coherente con la evidencia de la literatura de que la calidad de la especificación de entrada es el factor determinante de la coherencia de las pruebas generadas: el mejor desempeño se obtuvo precisamente en los requisitos funcionales, más estructurados, y el menor en las historias de usuario, más ambiguas [2], [16].

El posicionamiento de TDDMachine en la etapa *Red* del ciclo TDD la diferencia de herramientas como *ChatUniTest*, *TestSpark* o *Penguin*, orientadas a generar pruebas desde código fuente existente [24]–[26]. Al intervenir antes de que exista código de producción, la herramienta se alinea con el principio fundamental del TDD de escribir primero la prueba y, con ello, con la evidencia de que anticipar las pruebas reduce la densidad de defectos y el esfuerzo de corrección posterior a la entrega [6]–[8]. Los niveles de usabilidad ($SUS = 74,46$) y de aceptación (todos los constructos TAM por encima del umbral, con la Utilidad Percibida como predictor más fuerte de la Intención de Uso) indican, además, que la herramienta es percibida como útil y fácil de usar por usuarios sin experiencia previa en `pytest`, lo que aborda directamente la barrera de adopción del TDD asociada a la curva de aprendizaje del *testing* [10], [142], [143]. El puntaje SUS obtenido es comparable al de otras herramientas de soporte al desarrollo evaluadas con el mismo instrumento en la línea de investigación del grupo [139].

Entre las amenazas a la validez deben considerarse las siguientes. La validez externa está limitada por una muestra compuesta exclusivamente por estudiantes de un mismo semestre y por un conjunto de especificaciones que no cubre todos los dominios de aplicación posibles; la generalización a profesionales y a contextos industriales requiere estudios adicionales. La validez de constructo del criterio de viabilidad de ejecución está acotada por su evaluación

mediante inspección estructural del código y no mediante su ejecución efectiva, decisión coherente con el alcance de la herramienta —que cubre la etapa *Red* y no la *Green*— pero que impide afirmar la ejecutabilidad en *pipelines* reales de integración continua. Finalmente, las mediciones corresponden a una primera sesión de interacción, por lo que la estabilidad de las percepciones en un uso sostenido queda por confirmar.

VI. CONCLUSIONES

Este trabajo presentó TDDMachine, una herramienta CASE que automatiza la generación del estado inicial de las pruebas —la etapa *Red* del ciclo TDD— a partir de especificaciones estructuradas y mediante el modelo de lenguaje Claude 3.5 Sonnet, con validación sintáctica automática del código generado por análisis del AST de Python. La herramienta, cuyos requisitos se derivaron de una revisión sistemática de 108 estudios primarios y cuya pila tecnológica se seleccionó por evaluación multicriterio, produjo scripts `pytest` válidos en el 91,4 % de los intentos, alcanzó una calidad media evaluada por expertos de 4,08/5,0, obtuvo un puntaje SUS de 74,46/100 y superó el umbral de aceptación tecnológica en los cuatro constructos TAM. En conjunto, estos resultados validan la hipótesis central del proyecto: es posible construir una herramienta CASE que, a partir de especificaciones estructuradas y empleando un LLM como motor de generación y el AST como mecanismo de control de calidad, automatice la generación del estado inicial de pruebas para la metodología TDD.

En el plano teórico, el estudio conecta dos marcos previamente tratados por separado: el principio *test-first* del TDD —escribir la prueba antes que el código— y la teoría de aceptación tecnológica que explica la adopción de una herramienta a partir de su utilidad y facilidad de uso percibidas. Al situar la generación asistida por LLM en la etapa *Red* y evidenciar que esa anticipación es percibida como útil y usable, el trabajo aporta un puente empírico entre la anticipación de pruebas como práctica de calidad y la aceptación de las herramientas que la soportan. La principal contribución es, por tanto, empírica y metodológica: la evidencia de que anteponer una captura estructurada de requisitos a la generación por LLM, y verificar el resultado mediante AST, mejora la proporción de scripts válidos respecto de enfoques que operan sobre texto libre, y de que este proceso es percibido como útil y usable por desarrolladores noveles. Como líneas de trabajo futuro se plantean: la extensión de la validación mediante la ejecución real de los scripts en entornos controlados para medir la tasa de éxito en la etapa *Green*; la incorporación de instrucciones explícitas en los *prompts* para forzar escenarios negativos y de valor límite, atendiendo a la principal recomendación de los expertos; la ampliación de la evaluación a profesionales y a mayor diversidad de dominios; y la preparación de la versión en inglés del manuscrito para su envío a *Enfoque UTE*.

ETHICS AND INFORMED CONSENT

Both evaluation studies involved voluntary participants and followed an ethics protocol. Every participant provided written informed consent before any activity; the consent form stated the voluntary nature of participation, its exclusively academic purpose, the tasks to be performed, the data to be collected, the confidentiality guarantees, the right to withdraw at any time without penalty, and the data-retention period. The collected data were handled confidentially, stored in access-restricted digital forms, and reported only in aggregate, without individual identification.

FUNDING

This research received no external funding.

CONFLICT OF INTEREST

The authors declare that they have no conflict of interest.

ARTIFICIAL INTELLIGENCE STATEMENT

The authors declare that generative artificial intelligence tools were used to assist in language editing and in the drafting and structuring of this manuscript. The core scientific content, the design of the tool, the experimental design, the data collection and the analysis of results are the sole work and responsibility of the authors, who take full responsibility for the content of the manuscript.

REFERENCES

- [1] S. Lukaszczuk, F. Kroiß, and G. Fraser, “An empirical study of automated unit test generation for Python,” *Empirical Software Engineering*, vol. 28, no. 2, p. 36, 2023.
- [2] J. Wang, Y. Huang, C. Chen, Z. Liu, S. Wang, and Q. Wang, “Software testing with large language models: Survey, landscape, and vision,” *IEEE Transactions on Software Engineering*, vol. 50, no. 4, pp. 911–936, 2024.
- [3] M. Beller, G. Gousios, A. Panichella, and A. Zaidman, “When, how, and why developers (do not) test in their IDEs,” in *Proc. 10th Joint Meeting Found. Softw. Eng. (ESEC/FSE)*. New York, NY, USA: ACM, 2015, pp. 179–190.
- [4] P. Straubinger and G. Fraser, “A survey on what developers think about testing,” in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*. IEEE, 2023, pp. 80–90.
- [5] K. Beck, *Test-Driven Development: By Example*. Boston, MA, USA: Addison-Wesley, 2003.
- [6] N. Nagappan, E. M. Maximilien, T. Bhat, and L. Williams, “Realizing quality improvement through test driven development: results and experiences of four industrial teams,” *Empirical Software Engineering*, vol. 13, no. 3, pp. 289–302, 2008.
- [7] Y. Rafique and V. B. Mišić, “The effects of test-driven development on external quality and productivity: A meta-analysis,” *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 835–856, 2013.
- [8] H. Erdogmus, M. Morisio, and M. Torchiano, “On the effectiveness of the test-first approach to programming,” *IEEE Transactions on Software Engineering*, vol. 31, no. 3, pp. 226–237, 2005.
- [9] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018 — systems and software engineering — life cycle processes — requirements engineering,” International Standard, 2018.
- [10] M. Ghafari, T. Gross, D. Fucci, and M. Felderer, “Why research on test-driven development is inconclusive?” in *Proc. 14th ACM/IEEE Int. Symp. Empir. Softw. Eng. Meas. (ESEM)*. New York, NY, USA: ACM, 2020, pp. 1–10.
- [11] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
- [12] A. Carleton, D. Falessi, H. Zhang, and X. Xia, “Generative AI: Redefining the future of software engineering,” *IEEE Software*, vol. 41, no. 6, pp. 34–37, 2024.
- [13] J. Molina and J. Parraga-Alava, “Artificial neural networks for classification tasks: A systematic literature review,” *Enfoque UTE*, vol. 15, no. 4, pp. 1–10, 2024.
- [14] F. Carrillo, R. T. Tandazo, and L. B. Guaman, “Analysis of artificial intelligence methods for automatic bandwidth adjustment for wireless networks,” *Enfoque UTE*, vol. 16, no. 1, pp. 45–52, 2025.
- [15] J. B. Looor and J. Parraga-Alava, “Integration of IoT and artificial intelligence in ecuadorian agriculture: A systematic literature review (2020–2025),” *Enfoque UTE*, vol. 17, no. 1, pp. 12–22, 2026.
- [16] L. Yang *et al.*, “On the evaluation of large language models in unit test generation,” in *Proc. 39th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*. New York, NY, USA: ACM, 2024, pp. 1607–1619.
- [17] S. Rehan, B. Al-Bander, and A. A.-S. Ahmad, “Harnessing large language models for automated software testing: A leap towards scalable test case generation,” *Electronics*, vol. 14, no. 7, p. 1463, 2025.
- [18] S. Bhatia, T. Gandhi, D. Kumar, and P. Jalote, “Unit test generation using generative AI: A comparative performance analysis of autogeneration tools,” in *Proc. 1st Int. Workshop Large Lang. Models Code (LLM4Code)*. New York, NY, USA: ACM, 2024, pp. 54–61.
- [19] B. R. Korraprolu, P. Pinninti, and Y. R. Reddy, “Test case generation for requirements in natural language—an LLM comparison study,” in *Proc. 18th Innov. Softw. Eng. Conf. (ISEC)*. New York, NY, USA: ACM, 2025, pp. 1–5.
- [20] W. Takerngsaksiri, R. Charakorn, C. Tantithamthavorn, and Y. F. Li, “Pytester: Deep reinforcement learning for text-to-testcase generation,” *Journal of Systems and Software*, vol. 224, p. 112381, 2025.
- [21] N. S. Mathews and M. Nagappan, “Test-driven development for code generation,” in *Proc. 39th IEEE/ACM Int. Conf. Autom. Softw. Eng. (ASE)*. New York, NY, USA: ACM, 2024, pp. 1583–1594.
- [22] J. Mock and J. Melegati, “Generative AI for test driven development: Preliminary results,” in *Agile Processes in Software Engineering and Extreme Programming—Workshops (XP 2025)*. Cham, Switzerland: Springer Nature Switzerland, 2025, pp. 24–32.
- [23] J. C. Pancher, J. Melegati, and E. M. Guerra, “Exploratory test-driven development study with ChatGPT in different scenarios,” in *Agile Processes in Software Engineering and Extreme Programming (XP 2025)*. Cham, Switzerland: Springer Nature Switzerland, 2025, pp. 145–159.
- [24] Y. Chen, Z. Hu, C. Zhi, J. Han, S. Deng, and J. Yin, “ChatUniTest: A framework for LLM-based test generation,” in *Companion Proc. 32nd ACM Int. Conf. Found. Softw. Eng. (FSE)*. New York, NY, USA: ACM, 2024, pp. 572–576.
- [25] A. Sapozhnikov, M. Olsthoorn, A. Panichella, V. Kovalenko, and P. Derakhshanfar, “TestSpark: IntelliJ IDEA’s ultimate test generation companion,” in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.: Companion (ICSE-Companion)*. New York, NY, USA: ACM, 2024, pp. 30–34.
- [26] S. Lukaszczuk and G. Fraser, “Pynguin: Automated unit test generation for Python,” in *Proc. IEEE/ACM 44th Int. Conf. Softw. Eng.: Companion (ICSE-Companion)*. New York, NY, USA: ACM, 2022, pp. 168–172.
- [27] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, “Systematic literature reviews in software engineering—a systematic literature review,” *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, 2009.
- [28] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann *et al.*, “The PRISMA 2020 statement: an updated guideline for reporting systematic reviews,” p. n71, 2021.
- [29] R. Fazzolino and G. N. Rodrigues, “Feature-trace: Generating operational profile and supporting testing prioritization from bdd features,” in *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*, ser. SBES ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 332–336.
- [30] L. Campanile, M. S. d. Biase, S. Marrone, M. Raimondo, and L. Verde, “On the evaluation of bdd requirements with text-based metrics: The etcs-13 case study,” *Smart Innovation, Systems and Technologies*, vol. 309, pp. 561 – 571, 2022.
- [31] C. Wiecher, S. Japs, L. Kaiser, J. Greenyer, R. Dumitrescu, and C. Wolff, “Scenarios in the loop: Integrated requirements analysis and automotive system validation,” in *Proceedings - 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS-C 2020 - Companion Proceedings*. Association for Computing Machinery, Inc, 2020, pp. 199 – 208.
- [32] S. Tatale and V. Chandra Prakash, “Enhancing acceptance test driven development model with combinatorial logic,” *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 10, pp. 268 – 278, 2020.
- [33] T. Rocha Silva, M. Winckler, and H. Trætteberg, “Ensuring the consistency between user requirements and task models: A behavior-based automated approach,” *Proc. ACM Hum.-Comput. Interact.*, vol. 4, no. EICS, 2020.
- [34] T. N. Kudo, R. F. Bulcao-Neto, and A. M. R. Vincenzi, “A conceptual metamodel to bridging requirement patterns to test patterns,” in *33rd Brazilian Symposium on Software Engineering (SBES) / 10th Brazilian Conference on Software (CBSOFT)33rd Brazilian Symposium on Software Engineering (SBES) / 10th Brazilian Conference on Software (CBSOFT)*, 2019, pp. 155.0–160.0.
- [35] H. Bänder and H. Kuchen, “A model-driven approach for behavior-driven gui testing,” in *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1742–1751.
- [36] D. N. Jorge, P. D. L. Machado, E. L. G. Alves, and W. L. Andrade, “Integrating requirements specification and model-based testing in agile development,” in *26th IEEE International Requirements Engineering Conference (RE)26th IEEE International Requirements Engineering Conference (RE)*, 2018, pp. 336.0–346.0.
- [37] G. Lucassen, F. Dalpiaz, E. M. van der Werf, and S. Brinkkemper, “The use and effectiveness of user stories in practice,” in *22nd International Working Conference on Requirements Engineering - Foundation for Software Quality22nd International Working Conference on Requirements Engineering - Foundation for Software Quality*, vol. 9619.0, 2016, pp. 205.0–222.0.

- [38] A. Gupta and P. Bera, "A software tool to convert requirements to test cases," in *Proceedings of the 6th International Workshop on Requirements Engineering and Testing*, ser. RET '19. IEEE Press, 2019, p. 9–12.
- [39] S. Orekhov, P. Taran, and N. Bahatskyi, "An example of developing a high-level requirements specification for ai-based software using chatgpt," *International Journal of Wireless and Microwave Technologies*, vol. 16, no. 3, pp. 111 – 127, 2026.
- [40] G. Holodnik-Janczura, "The extension of user story template structure with an assessment question based on the kano model," in *37th International Conference on Information Systems Architecture and Technology (ISAT)* 37th International Conference on Information Systems Architecture and Technology (ISAT), vol. 523.0, 2017, pp. 137.0–150.0.
- [41] M. Ecar, F. Kepler, and J. P. S. da Silva, "Cosmic user story standard," in *19th International Conference on Agile Software Development (XP)* 19th International Conference on Agile Software Development (XP), vol. 314.0, 2018, pp. 3.0–18.0.
- [42] F. Pudlitz, F. Brokhausen, and A. Vogelsang, "What am i testing and where? comparing testing procedures based on lightweight requirements annotations," *EMPIRICAL SOFTWARE ENGINEERING*, vol. 25.0, no. 4.0, pp. 2809.0–2843.0, 2020.
- [43] A. Crapo, A. Moitra, C. McMillan, and D. Russell, "Requirements capture and analysis in assert(tm)," in *Proceedings - 2017 IEEE 25th International Requirements Engineering Conference, RE 2017*. Institute of Electrical and Electronics Engineers Inc., 2017, pp. 283 – 291.
- [44] B. Wei, "Requirements are all you need: From requirements to code with llms," in *32nd IEEE International Requirements Engineering Conference (RE)* 32nd IEEE International Requirements Engineering Conference (RE), 2024, pp. 416.0–422.0.
- [45] S. Sandhu, S. Rajendran, F. Sarracini, and M. Soleimani, "A framework for integrated behaviour driven development and model based feature design and verification," in *International Conference on Vehicle Technology and Intelligent Transport Systems, VEHTS - Proceedings*. Science and Technology Publications, Lda, 2026, p. 439 – 445.
- [46] C. Wiecher and J. Greenyer, "Besos: A tool for behavior-driven and scenario-based requirements modeling for systems of systems," in *CEUR Workshop Proceedings*, vol. 2857. CEUR-WS, 2021.
- [47] C. Wiecher, J. Greenyer, and J. Korte, "Test-driven scenario specification of automotive software components," in *ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)* ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C), 2019, pp. 12.0–17.0.
- [48] M. N. Zafar, W. Afzal, and E. Enoiu, "Towards a workflow for model-based testing of embedded systems," in *12th International Workshop on Automating TEST Case Design, Selection, and Evaluation (A-TEST)* 12th International Workshop on Automating TEST Case Design, Selection, and Evaluation (A-TEST), 2021, pp. 33.0–40.0.
- [49] M. N. Zafar, W. Afzal, E. Enoiu, A. Stratis, A. Arrieta, and G. Sagardui, "Model-based testing in practice: An industrial case study using graphwalker," in *Proceedings of the 14th Innovations in Software Engineering Conference (Formerly Known as India Software Engineering Conference)*, ser. ISEC '21. New York, NY, USA: Association for Computing Machinery, 2021.
- [50] W. Abdeen, X. Chen, and M. Unterkalmsteiner, "An approach for performance requirements verification and test environments generation," *REQUIREMENTS ENGINEERING*, vol. 28.0, no. 1.0, pp. 117.0–144.0, 2023.
- [51] S. Varpe, U. Mittal, R. K. Bhatia, and Ashpreet, "User acceptance test case generation using llms," in *2025 IEEE International Conference on Collaborative Advances in Software and COmputiNg (CASCON)*, 2025, pp. 532–538.
- [52] Z. A. Hamza and M. Hammad, "Generating test sequences from uml use case diagram: A case study," in *2020 Second International Sustainability and Resilience Conference: Technology and Innovation in Building Designs(51154)*, 2020, pp. 1–6.
- [53] E. M. Fredericks and B. H. C. Cheng, "Automated generation of adaptive test plans for self-adaptive systems," in *Proceedings of the 10th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*, ser. SEAMS '15. IEEE Press, 2015, p. 157–168.
- [54] I. Chernorutsky, P. Drobintsev, and V. Kotlyarov, "Industrial approach in requirement engineering," in *CEUR Workshop Proceedings*, vol. 1989. CEUR-WS, 2018, pp. 388 – 400.
- [55] K. Louzaoui and K. Benlhachmi, "Invalid input data and conformity testing for an object-oriented specification," in *2016 International Conference on Information Technology for Organizations Development, IT4OD 2016*. Institute of Electrical and Electronics Engineers Inc., 2016.
- [56] E. B. dos Santos, L. S. a. da Costa, B. S. Aragão, I. de Sousa Santos, and R. M. de Castro Andrade, "Extraction of test cases procedures from textual use cases to reduce test effort: Test factory experience report," in *Proceedings of the XVIII Brazilian Symposium on Software Quality*, ser. SBQS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 266–275.
- [57] P. Mahanto, S. K. Barisal, and D. P. Mohapatra, "Achieving mc/dc using uml communication diagram," in *Proceedings - 2018 International Conference on Information Technology, ICIT 2018*. Institute of Electrical and Electronics Engineers Inc., 2018, pp. 73 – 78.
- [58] G. K. Palshikar, N. Ramrakhiyani, S. Patil, S. Pawar, S. Hingmire, V. Varma, and P. Bhattacharyya, "Extraction of message sequence charts from software use-case descriptions," in *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, vol. 2. Association for Computational Linguistics (ACL), 2019, pp. 130 – 137.
- [59] A. Agrawal, B. Zhang, Y. Shivalingaiah, M. Vierhauser, and J. Cleland-Huang, "A requirements-driven platform for validating field operations of small uncrewed aerial vehicles," in *2023 IEEE 31st International Requirements Engineering Conference (RE)*, 2023, pp. 29–40.
- [60] W. S. Jang and R. Y. C. Kim, "Automatic generation mechanism of cause-effect graph with informal requirement specification based on the korean language," *Applied Sciences (Switzerland)*, vol. 11, no. 24, 2021.
- [61] R. Sinha, S. Patil, C. Pang, V. Vyatkin, and B. Dowdeswell, "Requirements engineering of industrial automation systems: Adapting the cesar requirements meta model for safety-critical smart grid software," in *IECON 2015 - 41st Annual Conference of the IEEE Industrial Electronics Society*. Institute of Electrical and Electronics Engineers Inc., 2015, pp. 2172 – 2177.
- [62] T. Yue, H. Zhang, S. Ali, and C. Liu, "A practical use case modeling approach to specify crosscutting concerns," in *Software Reuse: Bridging with Social-Awareness (ICSR 2016)*, LNCS, ser. Lecture Notes in Computer Science, vol. 9679. Cham: Springer, 2016, pp. 89–105.
- [63] T. Ishinari, K. Ueda, and Y. Shimizu, "Improving training data quality for automated test case generation using machine learning and its evaluation," in *CIIS 2025 - 2025 the 8th International Conference on Computational Intelligence and Intelligent Systems*. Association for Computing Machinery, Inc, 2025, pp. 44 – 51.
- [64] T. Bandyszak, M. Rzepka, T. Weyer, and K. Pohl, "Supporting the validation of structured analysis specifications in the engineering of information systems by test path exploration," in *ICEIS 2015 - 17th International Conference on Enterprise Information Systems, Proceedings*, vol. 2. SciTePress, 2015, pp. 252 – 259.
- [65] A. Ansari, M. B. Shagufta, A. Sadaf Fatima, and S. Tehreem, "Constructing test cases using natural language processing," in *Proceedings of the 3rd IEEE International Conference on Advances in Electrical and Electronics, Information, Communication and Bio-Informatics, AEEICB 2017*. Institute of Electrical and Electronics Engineers Inc., 2017, pp. 95 – 99.
- [66] W. Jang and R. Young Chul Kim, "Automatic test case generation mechanism with natural language-based korean requirement specifications," *IEEE Access*, vol. 13, pp. 177 305 – 177 317, 2025.
- [67] T. Diniz, E. L. Alves, A. G. Silva, and W. L. Andrade, "Reducing the discard of mbt test cases using distance functions," in *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*, ser. SBES '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 337–346.
- [68] A. Intana, K. Laosen, and T. Sriraksa, "An automated impact analysis approach for test cases based on changes of use case based requirement specifications," *INTERNATIONAL JOURNAL OF ADVANCED COMPUTER SCIENCE AND APPLICATIONS*, vol. 14.0, no. 1.0, pp. 967.0–980.0, 2023.
- [69] H. Zheng, J. Feng, W. Miao, and G. Pu, "Generating test cases from requirements: A case study in railway control system domain," in *Proceedings - 2021 International Symposium on Theoretical Aspects of Software Engineering, TASE 2021*. Institute of Electrical and Electronics Engineers Inc., 2021, pp. 183 – 190.

- [70] S. Liu and S. Nakajima, "Automatic test case and test oracle generation based on functional scenarios in formal specifications for conformance testing," *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 691–712, 2022.
- [71] S. Nogueira, H. L. S. Araujo, R. B. S. Araujo, J. Iyoda, and A. Sampaio, "Automatic generation of test cases and test purposes from natural language," *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, vol. 9526, pp. 145 – 161, 2016.
- [72] G. Carvalho, A. Cavalcanti, and A. Sampaio, "Modelling timed reactive systems from natural-language requirements," *Form. Asp. Comput.*, vol. 28, no. 5, p. 725–765, 2016.
- [73] F. Arruda, F. Barros, and A. Sampaio, "Deriving sound test scripts from requirements written in a controlled natural language," *Lecture Notes in Computer Science*, vol. 16363 LNCS, pp. 101 – 118, 2026.
- [74] J. Baxter, G. Carvalho, A. Cavalcanti, and F. R. Júnior, "Roboworld: Verification of robotic systems with environment in the loop," *Form. Asp. Comput.*, vol. 35, no. 4, 2023.
- [75] T. N. Kudo, R. d. F. Bulcão-Neto, V. V. G. Neto, and A. M. R. Vincenzi, "Aligning requirements and testing through metamodeling and patterns: design and evaluation," *Requirements Engineering*, vol. 28, no. 1, pp. 97 – 115, 2023.
- [76] A. Intana, K. Tantayakul, K. Laosen, and S. Charoenreh, "An approach of test case generation with software requirement ontology," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 8, pp. 1005 – 1014, 2023.
- [77] J. Miranda, A. C. R. Paiva, and A. R. da Silva, "Preliminary experiences in requirements-based security testing," *Communications in Computer and Information Science*, vol. 1266 CCIS, pp. 412 – 425, 2020.
- [78] N. Bhatia, J. Singh, and Vandana, "Automated test case generation from unstructured software requirements using advanced nlp techniques," in *2025 12th International Conference on Reliability, Infocom Technologies and Optimization, Trends and Future Directions, ICRITO 2025*. Institute of Electrical and Electronics Engineers Inc., 2025.
- [79] M. I. Malik, M. A. Sindhu, A. S. Khattak, R. A. Abbasi, and K. Saleem, "Automating test oracles from restricted natural language agile requirements," *Expert Systems*, vol. 38, no. 1, 2020.
- [80] T. Rocha Silva, "Towards a domain-specific language to specify interaction scenarios for web-based graphical user interfaces," in *Companion of the 2022 ACM SIGCHI Symposium on Engineering Interactive Computing Systems*, ser. EICS '22 Companion. New York, NY, USA: Association for Computing Machinery, 2022, p. 48–53.
- [81] T. R. Silva and B. Fitzgerald, "Empirical findings on bdd story parsing to support consistency assurance between requirements and artifacts," in *Proceedings of the 25th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 266–271.
- [82] J. Fischbach, M. Junker, A. Vogelsang, and D. Freudenstein, "Automated generation of test models from semi-structured requirements," in *Proceedings - 2019 IEEE 27th International Requirements Engineering Conference Workshops, REW 2019*. Institute of Electrical and Electronics Engineers Inc., 2019, pp. 263 – 269.
- [83] M. Osama, A. Zaki-Ismail, M. Abdelrazek, J. Grundy, and A. Ibrahim, "Srcm: A semi formal requirements representation model enabling system visualisation and quality checking," in *9th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)9th International Conference on Model-Driven Engineering and Software Development (MODELSWARD)*, 2021, pp. 278.0–285.0.
- [84] O. Olajubu, "Automated test case generation from domain-specific high-level requirement models," *Proceedings of the 2015 Conference on Research in Adaptive and Convergent Systems (RACS)*, 2018.
- [85] H. Liu, Y. Li, and Z. Li, "Ears2tf: A tool for automated planning test from semi-formalized requirements," in *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*. Knowledge Systems Institute Graduate School, 2022, pp. 469 – 470.
- [86] A. Veizaga, M. Alferez, D. Torre, M. Sabetzadeh, L. Briand, and E. Pitskhelauri, "Leveraging natural-language requirements for deriving better acceptance criteria from models," in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 218–228.
- [87] C. Wang, F. Pastore, A. Goknil, and L. C. Briand, "Automatic generation of acceptance test cases from use case specifications: An nlp-based approach," *IEEE Transactions on Software Engineering*, vol. 48, no. 2, pp. 585–616, 2022.
- [88] C. Wang, F. Pastore, A. Goknil, L. Briand, and Z. Iqbal, "Automatic generation of system test cases from use case specifications," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*, ser. ISSTA 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 385–396.
- [89] D. Clerissi, M. Leotta, G. Reggio, and F. Ricca, "Towards the generation of end-to-end web test scripts from requirements specifications," in *2017 IEEE 25th International Requirements Engineering Conference Workshops (REW)*, 2017, pp. 343–350.
- [90] P. X. Mai, F. Pastore, A. Goknil, and L. C. Briand, "Mcp: A security testing tool driven by requirements," in *Proceedings - 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion, ICSE-Companion 2019*. Institute of Electrical and Electronics Engineers Inc., 2019, pp. 55 – 58.
- [91] T.-B. Trinh and N.-T. Truong, "Tcg: an approach to improving test-driven development by exploiting use case specifications," *International Journal of System Assurance Engineering and Management*, vol. 17, no. 4, pp. 1159 – 1174, 2026.
- [92] L. Erazo, E. Martins, and J. G. Greggi, "Maritaca: from textual use case descriptions to behavior models," in *47th IEEE/IFIP Annual International Conference on Dependable Systems and Networks (DSN)47th IEEE/IFIP Annual International Conference on Dependable Systems and Networks (DSN)*, 2017, pp. 83.0–90.0.
- [93] S. Tiwari and A. Gupta, "An approach of generating test requirements for agile software development," in *ACM International Conference Proceeding Series*, vol. 18-20-February-2015. Association for Computing Machinery, 2015, pp. 186 – 195.
- [94] T.-B. Trinh, T.-V. Nguyen, N.-M. Le, and N. V. Ha, "Fstg: Few-shot test case generation from use case specifications using large language models," *Communications in Computer and Information Science*, vol. 2945 CCIS, pp. 302 – 316, 2026.
- [95] M. L. Roldán, M. Vegetti, S. Gonnet, H. Leone, and M. Marciszack, "An ontology for specifying and tracing requirements engineering artifacts and test artifacts," *CLEI Eletronic Journal (CLEIej)*, vol. 22, no. 1, 2019.
- [96] C. Wang, F. Pastore, A. Goknil, L. C. Briand, and Z. Iqbal, "Umtg: A toolset to automatically generate system test cases from use case specifications," in *2015 10th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering, ESEC/FSE 2015 - Proceedings*. Association for Computing Machinery, Inc, 2015, pp. 942 – 945.
- [97] S. Khamaiseh and D. Xu, "Software security testing via misuse case modeling," in *15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech)15th Intl Conf on Dependable, Autonomic and Secure Computing, 15th Intl Conf on Pervasive Intelligence and Computing, 3rd Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress(DASC/PiCom/DataCom/CyberSciTech)*, 2017, pp. 534.0–541.0.
- [98] C. Gralha, R. Pereira, M. Goulao, and J. Araujo, "On the impact of using different templates on creating and understanding user stories," in *Proceedings of the IEEE International Conference on Requirements Engineering*. IEEE Computer Society, 2021, pp. 209 – 220.
- [99] A. C. R. Paiva, D. Maciel, and A. R. da Silva, "From requirements to automated acceptance tests with the rsl language," in *14th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)14th International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*, vol. 1172.0, 2020, pp. 39.0–57.0.
- [100] A. C. Oran, E. Nascimento, G. Santos, and T. Conte, "Analysing requirements communication using use case specification and user stories," in *Proceedings of the XXXI Brazilian Symposium on Software Engineering*, ser. SBES '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 214–223.
- [101] H. Li, L. Zheng, and Q. Cai, "A test-driven approach for refining use case specifications of software requirements with llms," *Lecture Notes in Computer Science*, vol. 16229 LNCS, pp. 189 – 208, 2026.
- [102] T. Yue, S. Ali, and M. Zhang, "Rtcm: a natural language based, automated, and practical test case generation framework," in *Proceedings of the 2015 International Symposium on Software Testing and Analysis*,

- ser. ISSTA 2015. New York, NY, USA: Association for Computing Machinery, 2015, p. 397–408.
- [103] M. Zhang, T. Yue, S. Ali, B. Selic, O. Okariz, R. Norgre, and K. Intxausti, “Specifying uncertainty in use case models,” *Journal of Systems and Software*, vol. 144, pp. 573 – 603, 2018.
- [104] A. C. Corrêa Souza, F. L. S. Nunes, and M. E. Delamaro, “An automated functional testing approach for virtual reality applications,” *Software Testing Verification and Reliability*, vol. 28, no. 8, 2018.
- [105] I. L. Araújo, I. S. Santos, J. B. F. Filho, R. M. Andrade, and P. S. Neto, “Generating test cases and procedures from use cases in dynamic software product lines,” in *Proceedings of the ACM Symposium on Applied Computing*. Association for Computing Machinery, 2017, pp. 1296 – 1301.
- [106] S. Rodriguez, J. Thangarajah, and M. Winikoff, “A behaviour-driven approach for testing requirements via user and system stories in agent systems,” in *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems*, ser. AAMAS ’23. Richland, SC: International Foundation for Autonomous Agents and Multiagent Systems, 2023, p. 1182–1190.
- [107] J. Medeiros, A. Vasconcelos, C. Silva, and M. Goulão, “Requirements specification for developers in agile projects: Evaluation by two industrial case studies,” *Information and Software Technology*, vol. 117, 2019.
- [108] P. Baker, B. Manu, R. Moussa, and F. Sarro, “Industrial deployment of an ai multi-agent system for requirements-driven code verification,” in *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering*, ser. FSE Companion ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 485–495.
- [109] N. A. Moketar, M. Kamalrudin, S. Sidek, M. Robinson, and J. Grundy, “Testmereq: generating abstract tests for requirements validation,” in *Proceedings of the 3rd International Workshop on Software Engineering Research and Industrial Practice*, ser. SER&IP ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 39–45.
- [110] C. Wu, C. Wang, T. Li, and Y. Zhai, “A node-merging based approach for generating istar models from user stories,” in *Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE*. Knowledge Systems Institute Graduate School, 2022, pp. 257 – 262.
- [111] G. B. Herwanto, W. L. Muhamad, and W. Maaroja, “Enhancing automated user story quality assessment with large language models,” in *Proceedings of 2025 IEEE International Conference on Data and Software Engineering, ICoDSE 2025*. Institute of Electrical and Electronics Engineers Inc., 2025, pp. 196 – 201.
- [112] E. Dilorenzo, F. Ramos, D. Albuquerque, M. Perkusich, H. Almeida, and A. Perkusich, “Recommending test cases in agile software development with a taxonomy-guided knn approach,” in *Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1708–1716.
- [113] J. Frattini, J. Fischbach, and A. Bauer, “Cira: An open-source python package for automated generation of test case descriptions from natural language requirements,” in *31st IEEE International Requirements Engineering Conference (RE)31st IEEE International Requirements Engineering Conference (RE)*, 2023, pp. 68.0–71.0.
- [114] C. A. Martins, C. Dorneles, A. Barisic, T. R. Silva, and M. Winckler, “Cm-dir: A method to support the specification of the user’s dynamic behavior in recommender systems,” in *10th IFIP WG 13.2 International Working Conference on Human-Centered Software Engineering (HCSE)10th IFIP WG 13.2 International Working Conference on Human-Centered Software Engineering (HCSE)*, vol. 14793.0, 2024, pp. 26.0–46.0.
- [115] M. Nguyen, N. Hochgeschwender, and S. Wrede, “An analysis of behaviour-driven requirement specification for robotic competitions,” in *IEEE/ACM 5th International Workshop on Robotics Software Engineering (RoSE)IEEE/ACM 5th International Workshop on Robotics Software Engineering (RoSE)*, 2023, pp. 17.0–24.0.
- [116] E. C. dos Santos and P. Vilain, “Automated acceptance tests as software requirements: An experiment to compare the applicability of fit tables and gherkin language,” in *19th International Conference on Agile Software Development (XP)19th International Conference on Agile Software Development (XP)*, vol. 314.0, 2018, pp. 104.0–119.0.
- [117] J. Matula and F. Hunka, “Enterprise ontology-driven development,” *Lecture Notes in Business Information Processing*, vol. 332, pp. 3 – 15, 2018.
- [118] J. Saraiva and S. Soares, “Adoption of the lgpd inventory in the user stories and bdd scenarios creation,” in *Proceedings of the XXXVII Brazilian Symposium on Software Engineering*, ser. SBES ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 416–421.
- [119] —, “Privacy and security documents for agile software engineering: An experiment of lgpd inventory adoption,” in *2023 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, 2023, pp. 1–9.
- [120] M. França and F. Benitti, “Req2test: An approach for the automatic generation of unit tests from user stories using llms,” in *Proceedings of the 41st ACM/SIGAPP Symposium on Applied Computing*, ser. SAC ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1654–1663.
- [121] S. Charoenreh and A. Intana, “Enhancing software testing with ontology engineering approach,” in *2019 23rd International Computer Science and Engineering Conference (ICSEC)*, 2020, pp. 186–191.
- [122] R. Khaldi, A. Lehmann, B. Ghita, and U. Trick, “Agentic-ai framework for integrated design, implementation, testing, and operation of digital twin networks,” *IEEE Open Journal of the Communications Society*, vol. 7, pp. 4352–4375, 2026.
- [123] H.-V. Tran, N. P. Chi, and P. N. Hung, “Famres: A framework for automated testing of microservices based on requirement specifications,” *Lecture Notes in Networks and Systems*, vol. 1596 LNNS, pp. 356 – 366, 2025.
- [124] S. Morales, R. Clarisó, and J. Cabot, “A dsl for testing llms for fairness and bias,” in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems*, ser. MODELS ’24. New York, NY, USA: Association for Computing Machinery, 2024, p. 203–213.
- [125] D. H. Longo, P. Vilain, L. P. da Silva, and R. d. S. Mello, “A web framework for test automation: user scenarios through user interaction diagrams,” in *Proceedings of the 18th International Conference on Information Integration and Web-Based Applications and Services*, ser. iiWAS ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 458–467.
- [126] G. Fisher and C. Johnson, “Making formal methods more relevant to software engineering students via automated test generation,” in *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, ser. ITiCSE ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 224–229.
- [127] R. Venkatesh, U. Shrotri, A. Zare, and S. Agrawal, “Cost-effective functional testing of reactive software,” in *2015 International Conference on Evaluation of Novel Approaches to Software Engineering (ENASE)*, 2016, pp. 67–77.
- [128] S. Liu, “Validating formal specifications using testing-based specification animation,” in *Proceedings of the 4th FME Workshop on Formal Methods in Software Engineering*, ser. FormalISE ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 29–35.
- [129] J. Wen, L. Wu, X. Chen, X. Zhang, and X. Ma, “Research on formal specification generation methods for airborne software testing,” in *Proceedings of the 2026 6th International Conference on Computer Network Security and Software Engineering*, ser. CNSSE ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 184–189.
- [130] T. Ishinari and K. Ueda, “Data augmentation method for improving the accuracy of automated test case generation using attention seq2seq model,” in *2026 IEEE 2nd International Conference on Consumer Technology, ICCT-Pacific 2026*. Institute of Electrical and Electronics Engineers Inc., 2026, pp. 490 – 493.
- [131] N. Leon-Quinstedt, B. Lindgren, M. Yurdakul, and F. G. d. O. Neto, “Rest-at: An llm-based tool for automating traceability between requirements and test cases,” in *Proceedings of the 7th ACM/IEEE International Conference on Automation of Software Test*, ser. AST ’26. New York, NY, USA: Association for Computing Machinery, 2026, p. 1–11.
- [132] M. Motwani and Y. Brun, “Automatically generating precise oracles from structured natural language specifications,” in *Proceedings of the 41st International Conference on Software Engineering*, ser. ICSE ’19. IEEE Press, 2019, p. 188–199.
- [133] A. Fathima, K. Indulekha, and Shweta, “Automated detection and mitigation of requirement bad smells in software requirement specifications,” *Learning and Analytics in Intelligent Systems*, vol. 63, pp. 119 – 128, 2026.

- [134] M. Borg, J. Bengtsson, H. Österling, A. Hagelborn, I. Gagner, and P. Tomaszewski, "Quality assurance of generative dialog models in an evolving conversational agent used for swedish language practice," in *Proceedings of the 1st International Conference on AI Engineering: Software Engineering for AI*, ser. CAIN '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 22–32.
- [135] Y. Shimizu and K. Ueda, "Automatic test cases generation method by structuring specification documents using seq2seq," in *2025 1st International Conference on Consumer Technology, ICCT-Pacific 2025*. Institute of Electrical and Electronics Engineers Inc., 2025.
- [136] M. E. Salari, "Test automation for industrial programmable logic controller software: From requirements to executable tests," Ph.D. dissertation, Mälardalen University, Västerås, Sweden, 2026.
- [137] ISO/IEC/IEEE, "ISO/IEC/IEEE 12207:2017 — systems and software engineering — software life cycle processes," International Standard, 2017.
- [138] G. Guerrero-Ulloa, M. J. Hornos, and C. Rodríguez-Domínguez, "TDDM4IoT: A test-driven development methodology for internet of things (IoT)-based systems," in *Advances in Emerging Trends and Technologies (ICAT 2019)*, ser. Communications in Computer and Information Science, vol. 1193. Cham, Switzerland: Springer, 2020, pp. 41–55.
- [139] G. Guerrero-Ulloa, D. Carvajal-Suárez, P. Novais, M. J. Hornos, and C. Rodríguez-Domínguez, "Test-driven development tool for IoT systems," *IEEE Software*, vol. 42, no. 4, pp. 58–67, 2025.
- [140] G. Guerrero-Ulloa, C. Rodríguez-Domínguez, and M. J. Hornos, "Agile methodologies applied to the development of internet of things (IoT)-based systems: A review," *Sensors*, vol. 23, no. 2, p. 790, 2023.
- [141] J. Brooke, "SUS: A "quick and dirty" usability scale," in *Usability Evaluation in Industry*, P. W. Jordan, B. Thomas, B. A. Weerdmeester, and I. L. McClelland, Eds. London, U.K.: Taylor & Francis, 1996, pp. 189–194.
- [142] A. Bangor, P. Kortum, and J. Miller, "Determining what individual SUS scores mean: Adding an adjective rating scale," *Journal of Usability Studies*, vol. 4, no. 3, pp. 114–123, 2009.
- [143] F. D. Davis, "Perceived usefulness, perceived ease of use, and user acceptance of information technology," *MIS Quarterly*, vol. 13, no. 3, pp. 319–340, 1989.